ক্লাস উপকরণ · গভীরতা নমুনা

একজন ডেভেলপার এখানে কী কী ভাবে?

ক্লাস ১৯ · স্লাইস ১ — লগইন, প্রতিষ্ঠান, ভূমিকা
সিদ্ধান্ত, বিকল্প, প্রান্তিক কেস, ফাঁদ — সবকিছু

BTEB 28566 · Web Development Project

কোর্সের হিসাব

১০ পর্ব · ৩৬ ক্লাস

সিলেবাসের প্র্যাকটিক্যাল = পর্ব · প্রতিটি ক্লাস = একটি ডেক

মোট ৩৬টি ক্লাস — সবচেয়ে বেশি সময় পর্ব ৪ (কোডিং) আর পর্ব ২ (প্ল্যান)-এ



৩৬

ক্লাস · প্রতিটিতে ১৪–২২ স্লাইড

৭২

সিলেবাসের টপিক · একটাও বাদ নেই

১০

পর্ব · সিলেবাসের নম্বর অনুযায়ী

কেন কোডিং ৮ ক্লাস: স্কোপ সবচেয়ে বড় (৬ নম্বর, ৮ টপিক)। ভাগ: স্লাইস ১–৪ (৪ ক্লাস), ফ্রন্ট-ব্যাক (১), SEO (১), Git+PM (১), রিভিশন (১)।

ক্লাস ১৯ · সূচনা

কোড লেখার আগে সাতটি প্রশ্ন

এই সাতটির উত্তর না হলে লগইন সিস্টেম লেখা শুরু করবেন না

১ ইউজার কে — এক মানুষ, না এক প্রতিষ্ঠানের সদস্য?

✓ একই মানুষ দুই প্রতিষ্ঠানে থাকতে পারে

✕ না হলে মডেল ভাঙবে

২ ভূমিকা কোথায় থাকবে — ইউজারে, না সম্পর্কে?

✓ সম্পর্কে (Membership)

✕ ইউজারে রাখলে দ্বিতীয় প্রতিষ্ঠান সম্ভব নয়

৩ প্রতিষ্ঠান কোনটা — সেশন থেকে, না URL থেকে?

✓ URL-এর slug থেকে

✕ সেশন থেকে নিলে ভুল প্রতিষ্ঠানে কাজ হবে

৪ অনুমতি কে যাচাই করবে — টেমপ্লেট, না সার্ভার?

✓ সার্ভার (ডেকোরেটর)

✕ শুধু টেমপ্লেট হলে URL দিয়ে ঢোকা যাবে

৫ অন্যের ডেটা চাইলে ৪০৩, না ৪০৪?

✓ ৪০৪ — অস্তিত্ব গোপন রাখতে

✕ ৪০৩ বললে বোঝা যায় জিনিসটা আছে

৬ লগইনের পর কোথায় পাঠাব?

✓ সর্বোচ্চ ভূমিকার প্রতিষ্ঠানে

✕ বর্ণানুক্রমে প্রথম হলে ভুল ঘরে

৭ এই সিদ্ধান্তগুলো কীভাবে প্রমাণ করব?

✓ প্রতিটির জন্য একটি টেস্ট — মোট ১৩টি শেষে ১০৩

শিক্ষকের নোট: সাতটা প্রশ্ন ধরে ধরে জিজ্ঞেস করুন, উত্তর নিজে বলবেন না। ৪ ও ৫ নম্বরে সবাই ভুল করে — ওখানেই ক্লাসের আসল মূল্য।

সিদ্ধান্ত ১

ইউজার মডেল: ডিফল্ট User, না নিজের?

Django-র ডিফল্ট User-এ username থাকে — স্কুলে দরকার ইমেইল

বিকল্প	সুবিধা	সমস্যা	সিদ্ধান্ত
Django-র ডিফল্ট `User` ব্যবহার	দ্রুত, কিছু লিখতে হয় না	username বাধ্যতামূলক; পরে বদলানো প্রায় অসম্ভব	✕ **বাতিল**
ডিফল্ট User + Profile মডেল	নিরাপদ, কম কাজ	দুই টেবিলে ভাগ; লগইনে username চাইবে	~ আপস
কাস্টম User (email দিয়ে লগইন)	ইমেইলই পরিচয়; ভবিষ্যতে ফোন নম্বরও যোগ করা যাবে	প্রজেক্টের **প্রথম** মাইগ্রেশনে করতে হয়	✓ **বাছা হলো**

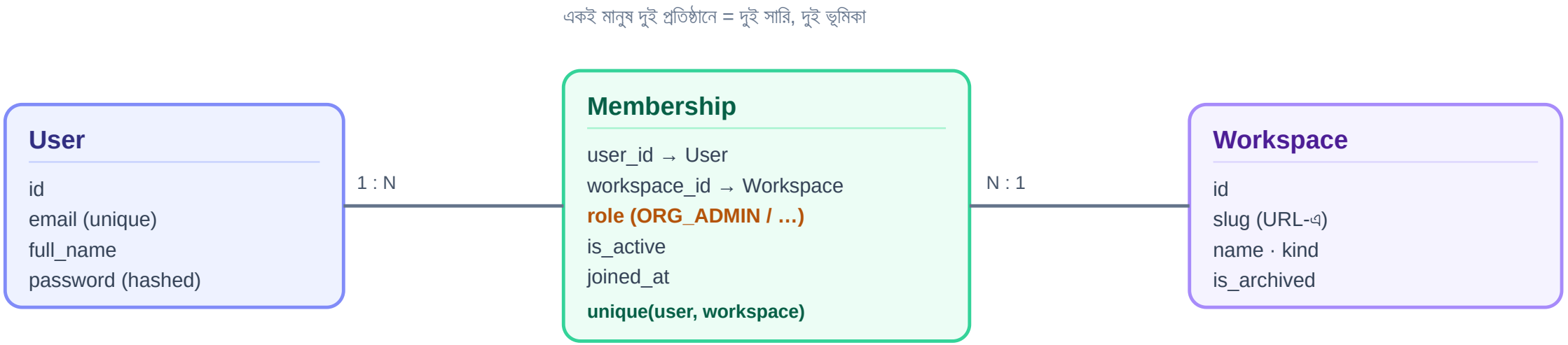
ফাঁদ: মাইগ্রেশন হয়ে যাওয়ার পরে কাস্টম User বানাতে চাইলে ডেটাবেস মুছতে হয়। `AUTH_USER_MODEL` সেট করতে হয় **প্রথম** `migrate`**-এর আগেই**।

বলুন: "এই ভুলটা প্রায় সবাই করে — শুরুতে সহজ মনে হয়, তিন মাস পরে ডেটাবেস নতুন করে বানাতে হয়।"

সিদ্ধান্ত ২

ভূমিকা কোথায় রাখব? — ERD

এক মানুষ, অনেক প্রতিষ্ঠান, প্রতিটিতে আলাদা ভূমিকা



যদি ভূমিকা User-এ রাখতাম

একটা `role` কলাম। শিক্ষক Sunrise-এ শিক্ষক, Springfield-এ কোঅর্ডিনেটর — কোনটা রাখবে?

Membership রাখলে

প্রতি জোড়ায় আলাদা সারি। একই মানুষ দুই জায়গায় দুই ভূমিকায়, দুটোই ঠিক।

বলুন: "ডেটাবেস ডিজাইনে সবচেয়ে বড় প্রশ্ন — কোন তথ্য কার গায়ে বসবে? এখানে ভূমিকা মানুষ আর প্রতিষ্ঠানের *সম্পর্কের* গুণ, কোনোটার নিজের নয়।"

সিদ্ধান্ত ৩

প্রতিষ্ঠান কোথা থেকে আসবে — সেশন, না URL?

এই একটা লাইনই পরবর্তী সব ডেটা লিক আটকাবে বা খুলে দেবে

সেশনে রাখলে ×

`/homework/5/` — প্রতিষ্ঠান অনুমান করতে হবে সেশন থেকে

ব্রাউজারে দুই ট্যাব খুললে, বা সেশন পুরোনো হলে → **ভুল প্রতিষ্ঠানের ডেটা**

URL-এ রাখলে ✓

`/w/sunrise-model-school/homework/5/`

প্রতিটি রিকোয়েস্টে প্রতিষ্ঠান স্পষ্ট — লুকানোর জায়গা নেই, ভুল হওয়ার সুযোগ নেই

নিয়ম: **রিসোর্স কোথায় আছে, তা URL-এই লেখা থাকুক** — সেশন শুধু "কে", URL বলে "কোথায়"।

প্রকৃত ঘটনা: এই প্রজেক্টেই একবার ভুল হয়েছিল — সাবস্ক্রিপশন গেট সেশন থেকে প্রতিষ্ঠান পড়ছিল, ফলে ভুল প্রতিষ্ঠানের মেয়াদ দেখে ভুল পর্দা দেখাত। ঠিক করতে হয়েছিল URL থেকে `slug` পড়ে।

কৌশল

একটি রিকোয়েস্টের যাত্রা

ব্রাউজার থেকে HTML পর্যন্ত — সাতটি ধাপ, প্রতিটির কাজ আলাদা



মনে রাখার নিয়ম: **যাচাই যত আগে, তত ভালো** — অনুমতি ব্যর্থ হলে কোয়েরিই চালানো হয় না।

বলুন: middleware-এর ক্রমটা বোর্ডে লিখুন: CustomDomain → CurrentWorkspace → SubscriptionGate। ক্রম বদলালে কী ভাঙবে সেটাও বলুন।

সূক্ষ্ম জিনিস

Middleware-এর ক্রম কেন এই ক্রম

ক্রম বদলালে সিস্টেম ভাঙে — কিন্তু এরর দেখায় না, চুপচাপ ভুল হয়

ক্রম	Middleware	কী করে	আগে বসালে কী হয়
১	`CustomDomainMiddleware`	কাস্টম ডোমেইন থেকে প্রতিষ্ঠান খুঁজে বের করে	—
২	`CurrentWorkspaceMiddleware`	URL-এর slug থেকে workspace নির্ধারণ করে	ডোমেইনের প্রতিষ্ঠান পাওয়া যেত না
৩	`SubscriptionGateMiddleware`	মেয়াদ শেষ হলে read-only করে দেয়	workspace না জানা থাকলে গেট কী যাচাই করবে?

গেট আগে বসালে

`request.workspace` তখনো `None` — মেয়াদ যাচাই হয় না, সব প্রতিষ্ঠান চিরকাল চলে। **নিরাপত্তা ফাঁক।**

সহজ পরীক্ষা

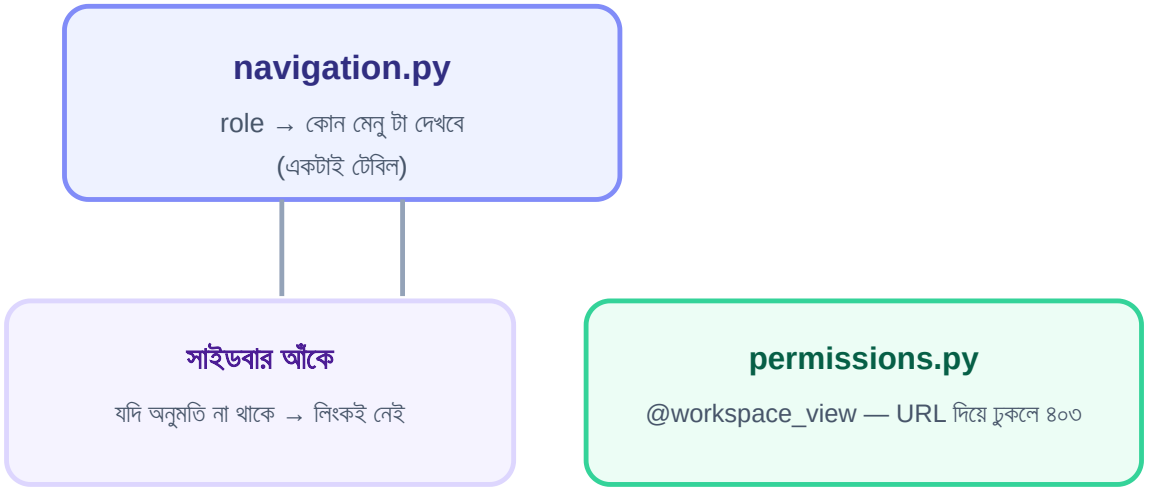
স্টেটমেন্টের ডেমোতে মেয়াদ শেষ করা একটা প্রতিষ্ঠান বানিয়ে দেখুন — read-only পর্দা আসছে কি না।

বলুন: "Middleware-এর ক্রম মানে নির্ভরতার ক্রম — পরে যেটা চলে, সেটা আগেরটার ফল ব্যবহার করে।"

সিদ্ধান্ত ৪

এক তালিকা, দুই জায়গায় ব্যবহার

মেনু আঁকা আর অনুমতি যাচাই — একই সত্য থেকে



যদি দুই জায়গায় আলাদা নিয়ম থাকত

মেনুতে লিংক নেই ✓

কিন্তু URL দিয়ে ঢোকা যায় ✗

= শিক্ষার্থী ইনভয়েস দেখে ফেলছে

এই ধরনের ফাঁক কোড রিভিউতে প্রায়ই ধরা পড়ে না,
কারণ পর্দায় সমস্যাটা দেখা যায় না।

বলুন: "নিজের প্রজেক্টে প্রশ্ন করুন — মেনুর তালিকা আর সিকিউরিটির তালিকা কি একই ফাইলে? না হলে আজই এক করে ফেলুন।"

কোড

ডেকোরেটর: ১২ লাইনে পুরো নিরাপত্তা

প্রতিটি workspace ভিউ-এর উপরে একটাই লাইন

```
def workspace_view(roles=None):
    def deco(view):
        @wraps(view)
        def inner(request, *a, **kw):
            ws = request.workspace          # URL থেকে
            if ws is None:
                raise Http404
            m = membership_for(request.user, ws)
            if m is None:
                raise Http404               # অস্তিত্ব গোপন
            if roles and m.role not in roles:
                raise PermissionDenied       # ৪০৩
            request.membership = m
            return view(request, *a, **kw)
        return inner
    return deco
```

ব্যবহার

`@workspace_view(["ORG_ADMIN","TEACHER"])`
— এক লাইনে ভূমিকা নির্ধারিত

দুটি আলাদা উত্তর

সদস্য নয় → **৪০৪** (জানতে পারবে না কিছু আছে)
সদস্য, কিন্তু ভূমিকা ভুল → **৪০৩**

বলুন: এই দুই লাইনের পার্থক্যটা জিজ্ঞেস করুন — `Http404` আর `PermissionDenied` কেন আলাদা। উত্তর: তথ্য ফাঁস ঠেকাতে।

কোড

QuerySet কাটা — ভুলে গেলেই লিক

অনুমতি ঠিক থাকলেও কোয়েরি ভুল হলে অন্য প্রতিষ্ঠানের ডেটা আসে

ভুল ×

```
Student.objects.get(pk=st_id)
```

যেকোনো প্রতিষ্ঠানের শিক্ষার্থী আসবে — আইডিটা জানলেই যথেষ্ট।

ঠিক ✓

```
Student.objects.get(
  pk=st_id, workspace=request.workspace)
```

প্রতিটি কোয়েরিতে workspace — না মিললে `DoesNotExist` → 404।

নিয়ম: **প্রতিটি মডেলে `workspace` ফিল্ড**, আর **প্রতিটি কোয়েরিতে সেটার শর্ত**। ব্যতিক্রম শুধু আপনার প্ল্যাটফর্ম প্যানেল।

বলুন: "এই ভুলটা সবচেয়ে বেশি হয় — কারণ অনুমতি ঠিক থাকে, তাই টেস্টেও ধরা পড়ে না যদি না আলাদা করে টেস্ট লেখেন।"

ডেভেলপারের চেকলিস্ট

যে ১০টি প্রান্তিক অবস্থার কথা ভাবতে হয়

এগুলো আগে না ভাবলে পরে বাগ — আর বাগ খুঁজতে সময় বেশি লাগে

#	অবস্থা	সিদ্ধান্ত
১	একই মানুষ দুই প্রতিষ্ঠানে	দুই Membership — লগইনে সর্বোচ্চ ভূমিকারটা আগে
২	প্রতিষ্ঠান বাতিল (archived) কিন্তু সদস্য এখনো আছেন	ভেতরে ঢোকা যাবে না; সুপারইউজার ছাড়া
৩	শিক্ষককে দুই ক্লাসে যোগ করা হয়েছে	একই প্রতিষ্ঠানে, দুই assignment ভিন্ন
৪	মেয়াদ শেষ, কিন্তু ব্যবহারকারী ফর্ম খুলে রেখেছেন	POST-ও আটকাবে — read-only-তে লেখা যায় না
৫	URL-এ ভুল slug	৪০৪, লগইন পর্দায় পাঠানো নয়
৬	সেশন পুরোনো, নতুন প্রতিষ্ঠান যোগ হয়েছে	লগইনে পুরোনো workspace সেশন থেকে মুছে ফেলা হয়
৭	একই ইমেইলে দুইবার সাইনআপ	Membership-এ `unique(user, workspace)` + ইমেইল unique
৮	সদস্য বাদ দেওয়ার পর তার পুরোনো ডেটা	ডেটা থাকে, অ্যাক্সেস যায় — ডিলিট নয়
৯	সুপারইউজার গেট বাইপাস করছিল	বাইপাস সরানো, বদলে allow-list পথ
১০	ব্যবহারকারী নিজেই একাধিক ভূমিকা চাইলে	ব্লক — এক প্রতিষ্ঠানে একজন এক ভূমিকা

বলুন: "এই তালিকাটাই একজন সিনিয়র আর জুনিয়র ডেভেলপারের পার্থক্য — ১, ৪, ৮ নম্বর আগে থেকে ভাবা মানে তিনটা বাগ জন্মানোর আগেই ঠেকানো।"

প্রমাণ

প্রতিটি সিদ্ধান্তের একটি টেস্ট

স্লাইস ১ শেষে ১৩টি টেস্ট — আজ ১০৩

যা যাচাই হচ্ছে	কীভাবে	ফল
ইমেইল দিয়ে লগইন হয়	`client.login(email=…)`	✓ পাস
ভুল পাসওয়ার্ড আটকায়	ভুল পাসওয়ার্ড পোস্ট	✓ পাস
সদস্য নয় → ৪০৪	অন্য প্রতিষ্ঠানের slug	✓ পাস
ভূমিকা ভুল → ৪০৩	শিক্ষক হিসেবে বিলিং URL	✓ পাস
একই প্রতিষ্ঠানে দ্বিতীয়বার যোগ دادن	`IntegrityError` আশা	✓ পাস
লগইন সঠিক প্রতিষ্ঠানে নামায়	admin → sunrise, springfield@ → springfield	✓ পাস
মেয়াদ শেষ → read-only	ট্রায়াল শেষ করা প্রতিষ্ঠান	✓ পাস

```
python manage.py test --settings=config.settings.sqlite
Ran 103 tests in 50.9s — OK
```

বলুন: "টেস্ট লেখার সময় ভাবুন — কোন ভুলটা হলে আমি ঘুমাতে পারব না? সেই ভুলটার টেস্ট আগে লিখুন।"

১৯

এই ক্লাসের পাঁচটি শিক্ষা

① ভূমিকা মানুষ আর প্রতিষ্ঠানের **সম্পর্কের** গুণ, কোনোটার নিজের নয়

② প্রতিষ্ঠান আসবে **URL** থেকে, সেশন থেকে নয়

③ অনুমতি যদি ব্যর্থ হয়, ডেটাবেসে **পৌঁছাবে না**

④ সদস্য নয় → ৪০৪ · ভূমিকা ভুল → ৪০৩

⑤ প্রতিটি সিদ্ধান্তের একটি **টেস্ট** থাকবে

পরের ক্লাস

ক্লাস ২০ · স্লাইস ২

শিক্ষাবর্ষ, ক্লাস, শিক্ষার্থী ভর্তি — আর ডেটাবেসেই বাধা

শিক্ষাবর্ষ → টার্ম → ক্লাস

কেন বছর আর টার্ম আলাদা টেবিল — এক বছরে তিনটি টার্ম, ফলাফল আলাদা।

ভর্তি (Enrollment)

শিক্ষার্থী আর ক্লাসের মধ্যে সম্পর্ক — মাঝের টেবিল কেন দরকার।

ডেটাবেসেই বাধা

একই ছাত্র দুইবার সক্রিয় ভর্তি হতে পারবে না — partial unique index।

আগের ক্লাসের **তিনটি সিদ্ধান্ত** এখানে কাজে লাগবে — Membership-এর মতো Enrollment-ও একটি সম্পর্ক, আর সেটাও workspace দিয়ে কাটা হবে।

শিক্ষকের নোট: ক্লাস শেষে ২ মিনিট: "আজকের কোন সিদ্ধান্তটা তোমার প্রজেক্টেও প্রযোজ্য?" — হাত তুলে ২–৩ জন বলবে।